DUBLIN CITY UNIVERSITY

SEMESTER 1 EXAMINATIONS 2015/2016

MODULE: CA320 - Computability and Complexity

PROGRAMME(S):

CASE - BSc in Computer Applications (Sft.Eng.)
CPSSD - BSc in ComputationalProblem SolvandSW Dev.

YEAR OF STUDY: 3

EXAMINERS: Dr David Sinclair (Ph:5510)

TIME ALLOWED: 2 hours

INSTRUCTIONS: Answer 4 Questions.
At least 1 question must be from Section A.
All questions carry equal marks.

PLEASE DO NOT TURN OVER THIS PAGE UNTIL INSTRUCTED TO DO SO

The use of programmable or text storing calculators is expressly forbidden.
Please note that where a candidate answers more than the required number of questions, the examiner will mark all questions attempted and then select the highest scoring ones.

Requirements for this paper (Please mark (X) as appropriate)

☐	*Log Tables*	☐	*Thermodynamic Tables*
☐	*Graph Paper*	☐	*Actuarial Tables*
☐	*Dictionaries*	☐	*MCQ Only - Do not publish*
☐	*Statistical Tables*	☐	*Attached Answer Sheet*

Section A

QUESTION 1 ***[Total marks: 25]***

1(a) [5 Marks]

Describe *list comprehensions* in the Haskell Programming language using examples where necessary.

1(b) [8 Marks]

Write a Haskell function called `vowels` that take a string and *using list comprehension* returns a string containing all the vowels from the original string. Include the function type for `vowels`. For example, `vowels "Hello World"` would evaluate to `"eoo"`.

1(c) [12 Marks]

Without using the list concatenation operator $++$, write a Haskell function that takes a list of lists as input and returns a list that is the concatenation of all the original lists. For example, if the input to the function was `[[3,2,1],[8,9],[1,2,3,4]]`, the function would evaluate to `[3,2,1,8,9,1,2,3,4]`.

[End Question 1]

QUESTION 2 ***[Total marks: 25]***

2(a) [5 Marks]

Describe function types in Haskell and the role that *class constraints* play in function types.

2(b) [10 Marks]

Write a Haskell program that takes a list of numbers (integer and floating point numbers) and returns the average of the non-negative numbers minus the average of the negative numbers. Consider 0 as a non-negative number.

2(c) [10 Marks]

Without using the built-in function `reverse`, write a Haskell function that tests if an array of elements is a palindrome, i.e. the reverse of the array is equal to the original array. Include its function type.

[End Question 2]

Section B

QUESTION 3 ***[Total marks: 25]***

3(a) [5 Marks]

Briefly describe the difference between *computability* and *complexity*.

3(b) [5 Marks]

In the study of Computability & Complexity, why do we focus on languages?

3(c) [15 Marks]

What is a *Universal Turing Machine*? What is its significance?

[End Question 3]

QUESTION 4 ***[Total marks: 25]***

4(a) [5 Marks]

Formally define a *context-free grammar*.

4(b) [10 Marks]

Formally describe the simplest automaton that can recognise a context-free grammar.

4(c) [10 Marks]

Construct an automaton (of the type indicated in part (b)) that recognises the language $L = \{ww^r | w^r$ is the reverse of the string $w\}$ over the alphabet $\Sigma = \{a, b, c\}$.

[End Question 4]

QUESTION 5 ***[Total marks: 25]***

5(a) [5 Marks]

Describe the complexity classes $\mathbf{P}$ and $\mathbf{NP}$ and what is the relationship between them.

5(b) [10 Marks]

Prove that a language L belongs to $\mathbf{NP}$ if and only if a language $L_{check} \in \mathbf{P}$ and a polynomial p exists such that

$$L = \{x | \exists y.x, y \in L_{check} \wedge y \leq p(x)\}$$

5(c) [10 Marks]

Describe a problem that belongs to the class $\mathbf{NP}$ and, using the theorem in part(b) of this question, prove that it is a $\mathbf{NP}$ problem.

[End Question 5]

[END OF EXAM]